At a high level, Docker provides five capabilities:

- *Packaging.* The ability to package software into a reusable, shareable format known as *images*.
- *Distribution.* The ability to easily share packaged software (images) with other people and deploy it to different machines.
- *Runtime.* The ability to run, pause, restart, or stop packaged software in a reliable, repeatable way.
- *Infrastructure creation.* Creating virtual machines ready to run our Docker containers.
- *Orchestration and scaling.* Managing the release of software to a single Docker node or across an entire cluster.

Together, these five things combine to enable a new way of delivering and running software.

Why Use Docker?

To build a Rails app, we typically develop on our local machine. Rather than each team member manually maintaining their own local development environments, we can use Docker to provide a common, standardized environment. This saves on repeated effort and helps avoid many forms of the "works on my machine" issues that can waste hours.

Other benefits of using Docker for your development environment include:

- *A holistic view of your app.* Rails apps typically need a database and other external dependencies like Redis and Elasticsearch. With Docker, these dependencies are no longer an afterthought or "add-on" like in Heroku; they are described and managed as fundamental parts of your app.
- *Single-command app installation and setup.* Have you ever set up a Rails app on your machine and spent an excessive amount of time installing specific versions of its software dependencies? Docker's built-in delivery mechanism means that new team members can go from zero to a running app in minutes. No laborious, error-prone, manual setup steps here.
- *Easy version management of dependencies.* Want to make sure everything works before switching to a new version of Ruby or upgrading the database? No problem: running containers is cheap. Just change the image version and away you go.